

Question 1 (5 marks)

1.a. You are training a language model on a highly structured dataset consisting only of standard traffic light sequences (e.g., "Red, Green, Yellow, Red..."). After millions of epochs, what is the theoretical lower bound (in bits) of the cross-entropy loss your model can achieve on this dataset, and why?

1.b. Using $H(P,Q) = -\sum p(x)\log_2 q(x)$, give a short argument for why $H(P,Q)$ can never be smaller than $H(P)$. What would have to be true about Q for equality to hold?

Answer Key:

1.a. Cross-Entropy Lower Bound (2 Marks)

- **Target Answer (1 Mark):** The theoretical lower bound is **0 bits**.
- **Justification (1 Marks):** The dataset is strictly deterministic (e.g., Red is always followed by Green). Because there is zero uncertainty in the true data distribution, the true entropy $H(P)$ is exactly 0. Cross-entropy is bounded by true entropy.
- **Note:** If students write that the bound is greater than zero due to model imperfections, deduct 1.5 marks. The question asks for the *theoretical* lower bound.

Alternate Accepted Answer (1 Mark): $\log_2(3)$ bits (or ~ 1.58 bits).

- **Justification (1 Mark):** Because the prompt's sequence ("Red, Green, Yellow, Red...") truncated before showing the next "Green", the student assumed a uniform random distribution across the 3 traffic light states rather than a deterministic cycle.

Note: Award full marks (2/2) for **either** answer, provided their justification matches their mathematical conclusion. Award 0 marks if a student claims the bound is greater than zero due to "model imperfections." This shows they are answering a practical deep learning question rather than the theoretical question asked.

1.b. Intuitive Argument for $H(P,Q) \geq H(P)$ (3 Marks)

- **Target Answer (Simpler Intuitive Version) (2 Marks):** Entropy $H(P)$ represents the most mathematically efficient way to encode data using the true probabilities. Cross-entropy $H(P,Q)$ is the bits used when we encode based on a flawed guess (distribution Q). Using a flawed guess means we are assigning suboptimal code lengths, which inherently wastes bits. Therefore, it will always require equal or more bits than the theoretical optimum.
- **Alternative Answer 1 (Information Theory / KL Divergence):** Cross-entropy is mathematically defined as the true entropy plus the Kullback-Leibler (KL) Divergence:

$H(P,Q) = H(P) + D_{\{KL\}}(P||Q)$. Since KL Divergence measures "wasted bits" and is strictly non-negative ($D_{\{KL\}} \geq 0$), $H(P,Q)$ must be greater than or equal to $H(P)$.

- **Alternative Answer 2 (Gibbs' Inequality):** Students may directly cite Gibbs' Inequality, which mathematically proves that $-\sum p(x) \log_2 q(x) \geq -\sum p(x) \log_2 p(x)$. But explanation needed
- **Equality Condition (1 Mark):** Equality holds if and only if the predicted distribution perfectly matches the true distribution ($Q = P$ for all events).

[1.b Alternative Grading Key: "Single Label" Proof (3 Marks)]

If attempts the single-label proof, evaluate their answer using this checklist:

1. The Base Proof & Assumptions (1 Mark)

- **Assumption (0.5 M):** Must explicitly state the constraint: true distribution has one label where $p(x) = 1$, and all others are 0.
- **Execution (0.5 M):** Shows true entropy $H(P) = 0$ and cross-entropy $H(P,Q) = -1 \log_2(q(x))$. Because a probability $q(x) \leq 1$, the term $-\log_2(q(x))$ must be ≥ 0 .

2. Generalization (1 Mark)

- The student must explain how this applies when $\sum p(x) = 1$. Award the mark for either approach:
- *Conceptual:* If Q is wrong, it inevitably assigns lower probabilities to high-frequency events. This inflates the penalty $-\log_2(q(x))$ for those common events, guaranteeing a higher total average.
- *Mathematical:* Sets up the difference as $\sum p(x) \log_2(p(x)/q(x))$ and states it is mathematically ≥ 0 (e.g., citing Jensen's inequality or Gibbs' inequality).

3. Equality Condition (1 Marks)

- Must state that the predicted distribution Q must exactly equal the true distribution P for every label (for the single-label case, $q(x) = 1$).

Question 2 (5 marks)

You are deploying a Locality Sensitive Hashing deduplication pipeline. You have a strict memory budget that limits you to storing a signature of exactly **64 total hash values** per document.

You must choose between two bucket configurations:

Configuration A: 32 bands of 2 rows.

Configuration B: 8 bands of 8 rows.

- Which configuration is too lenient and will produce a massive amount of **false positives**?
- Which configuration is very strict and will fail to catch actual near-duplicates (**false negatives**)?
- Prove your qualitative reasoning by calculating the approximate similarity threshold value (s^*) for both configurations.

Answer Key:

2.a. Lenient/Massive False Positives (1 Mark)

- Answer:** Configuration A.
- (Note: Award 1 mark purely for identifying the correct configuration).*

2.b. Strict/Massive False Negatives (1 Mark)

- Answer:** Configuration B.
- (Note: Award 1 mark purely for identifying the correct configuration).*

2.c. Calculation with the right formula(2 Marks)

- Config A Calculation (1 Mark):** $s^* \approx (1/32)^{1/2} \approx 0.176$ (Accept 17% to 18%, or the unsimplified fraction/square root).
- Config B Calculation (1 Mark):** $s^* \approx (1/8)^{1/8} \approx 0.75$ (Accept ~75%, or the unsimplified fraction/root).

General Explanation & Qualitative Reasoning (1 Mark)

- Criteria:** The student must conceptually explain *why* Config A is lenient (e.g., only 2 rows makes it highly probable for dissimilar documents to match) OR *why* Config B is strict (e.g., requiring 8 exact matches in a band means documents must be virtually identical).
- Note:** This is a "floating" mark. It does not need to be in Part C. Award this 1 mark if the sufficient reasoning is written *anywhere* in the student's response for Question 2 (whether under a, b, or c).

Question 3 (4 marks)

Consider a 4-word vocabulary for weather: "rainy", "cloudy", "windy", and "sunny". The true probability distribution y_{true} based on historical data is: $y_{rainy} = \frac{1}{2}$, $y_{cloudy} = \frac{1}{4}$, $y_{windy} = \frac{1}{4}$, and $y_{sunny} = 0$.

- Model A** predicts a uniform distribution across the 4 weather types.

- **Model B** is highly confident it will be sunny, predicting $p_{sunny} = \frac{29}{32}$, and distributing the rest equally for "rainy", "cloudy", and "windy".

Calculate the cross-entropy loss for both models using y_{true} . Which model incurs a larger penalty, and what does this demonstrate about language models?

Answer Key:

Context: Cross-entropy loss calculation on a 4-word vocabulary using $y_{true}(y_{rainy}=1/2, y_{cloudy}=1/4, y_{windy}=1/4, y_{sunny}=0)$.

Model A Calculation (1.5 Marks) Model A predicts a uniform distribution, so $p = 1/4$ for all classes.

$$CE_A = - (\frac{1}{2} \log_2(\frac{1}{4}) + \frac{1}{4} \log_2(\frac{1}{4}) + \frac{1}{4} \log_2(\frac{1}{4}) + 0)$$

$$CE_A = - (\frac{1}{2} (-2) + \frac{1}{4} (-2) + \frac{1}{4} (-2)) = - (-1 - 0.5 - 0.5) = 2 \text{ bits}$$

Model B Calculation (1.5 Marks) Model B predicts $p_{sunny} = 29/32$. The remaining $3/32$ is distributed equally, so $p_{rainy}, p_{cloudy}, p_{windy}$ each equal $1/32$.

$$CE_B = - (\frac{1}{2} \log_2(1/32) + \frac{1}{4} \log_2(1/32) + \frac{1}{4} \log_2(1/32) + 0)$$

$$CE_B = - (\frac{1}{2} (-5) + \frac{1}{4} (-5) + \frac{1}{4} (-5)) = - (-2.5 - 1.25 - 1.25) = 5 \text{ bits}$$

Conclusion (1 Mark)

Model B incurs a larger penalty. This demonstrates that models are heavily penalized for being highly confident in incorrect predictions (assigning extreme low probability to the actual true events).

Question 4 (8 marks)

You are evaluating a tokenizer on the following input string: z_yz99_x

Stage 1: Regex Pre-tokenization: The raw text is strictly split into independent, non-crossing chunks using this exact regular expression: `\d+[a-z]+|_(?=[a-z])|_+`

(Regex interpretation: a chunk of digits OR a chunk of lowercase letters OR a single underscore OR consecutive underscores).

Stage 2: BPE Inference: Each chunk from Stage 1 is processed independently. The base vocabulary consists of single characters. The tokenizer learned exactly two merge rules, which must be executed in this strict priority order:

1. $(y, z) \rightarrow yz$

2. $(_, x) \rightarrow _x$

- Perform the Stage 1 regex execution on the input string. Output the exact sequence of chunks, separated by a comma (e.g., chunk1, chunk2). Briefly justify how the regex evaluated the different underscores.
- Perform Stage 2 on your isolated chunks. Show the step-by-step merge sequence for each chunk. State the final token sequence and the total token count.

Answer Key:

The Final Chunks (1 Mark): z, $_$, yz, 99, $_$, x

Step-by-step Chunking Process(2 Marks): Look for a trace of the regex $\backslash d+| [a-z]+| _ (?=[a-z]) _ +$ across z_yz99_x:

- **Step 1:** Reads z. Matches $[a-z]^+$ (lowercase letters). **Chunk 1: z**
- **Step 2:** Hits $_$. Evaluates $_ (?=[a-z])$. It looks ahead, sees y, and condition met.
 - *Note (Lookahead vs. Normal ?):* $(?=...)$ is a zero-width lookahead. It verifies the letter exists but **does not consume it**, correctly isolating $_$. A normal optional $?$ (e.g., $_ ?[a-z]^+$) would consume the letters, wrongly creating the chunk $_ yz$.
 - **Chunk 2:** $_$
- **Step 3:** Resumes at y. Matches $[a-z]^+$. **Chunk 3: yz**
- **Step 4:** Hits 9. Matches $\backslash d^+$ (digits). **Chunk 4: 99**
- **Step 5:** Hits $_$. Evaluates $_ (?=[a-z])$, sees x ahead, and isolates $_$ just like Step 2.
Chunk 5: $_$
- **Step 6:** Resumes at x. Matches $[a-z]^+$. **Chunk 6: x**

4.b. Stage 2 BPE Inference (5 Marks)

- Valid Merge Rule 1 (1 Mark): The chunk yz successfully triggers Rule 1 $((y,z) \rightarrow yz)$.
- The Boundary Block / Rule 2 Failure (2 Marks): Rule 2 $((_,x) \rightarrow _x)$ fails. Stage 1 separated $_$ and x into independent chunks, and BPE cannot cross chunk boundaries to merge them. (Deduct 2 marks immediately if they merge $_x$).
- Final Output (1 Mark): **z, $_$, yz, 9, 9, $_$, x**
- Token Count (1 Mark): 7 tokens

Question 5 (4 marks)

You use a deduplication filter that splits text into overlapping 3-word shingles (sliding windows).

A spammer tries to bypass your filter using a 300-word document (Doc A). To avoid detection, they replace exactly every 3rd word with a synonym (Word 3, 6, 9, etc.), leaving the rest untouched. This creates Doc B. The spammer assumes Doc B will retain a 66% similarity because only 1/3 of the words changed.

1. How many original 3-word shingles from Doc A survive untouched in Doc B?
2. What is the true Jaccard similarity (from 0 to 1) for the 3-word shingles between Doc A and Doc B? Explain your answers.

Answer Key:

Context: 3-word overlapping shingles on a 300-word document, where every 3rd word is replaced to bypass a filter.

1. Surviving Original Shingles (2 Marks)

- **Answer: 0** (Zero).
- **Explanation:** Overlapping 3-word sliding windows function sequentially (Words 1-2-3, 2-3-4, 3-4-5, 4-5-6, etc.). Because every set of three consecutive integers contains exactly one multiple of 3, every single shingle in the document contains at least one replaced word. None survive untouched.

2. True Jaccard Similarity (2 Marks)

- **Answer: 0** (or 0%).
- **Explanation:** Jaccard similarity is calculated as Intersection/Union. Because there are zero surviving identical shingles between Doc A and Doc B, the intersection is 0, resulting in a true similarity score of 0.
- **Note:** The spammer's assumption of 66% similarity is a decoy based on word-level edits rather than structural n-gram mechanics.

Question 6 (4 marks)

The original 2017 Transformer applied Layer Normalization *after* the residual addition (Post-LN: $x = \text{LayerNorm}(x + \text{Sublayer}(x))$). Modern LLMs almost universally apply it *before* the sublayer (Pre-LN: $x = x + \text{Sublayer}(\text{LayerNorm}(x))$).

If you train a 100-layer Post-LN model without a careful learning rate warmup, it often fails to converge. A 100-layer Pre-LN model converges smoothly without the warmup.

What mathematical property of the residual stream in Pre-LN prevents the vanishing gradient problem at high depths?

Answer Key:

In the Pre-LN architecture ($x = x + \text{Sublayer}(\text{LayerNorm}(x))$), the Layer Normalization operation is moved inside the residual branch, meaning it is applied *before* the sublayer transformations. This structural change removes normalization from the main residual pathway entirely. As a result, the main residual stream becomes a pure, continuous chain of linear addition operations. Because the derivative of addition is exactly 1, error gradients can flow directly backward through this identity path during backpropagation without being repeatedly scaled down or squashed by normalization layers, directly preventing the vanishing gradient problem at high depths.

Key Points (Award 1-1.5 Marks per point hit):

- **Point 1:** The student must state that LayerNorm is removed from the main residual pathway and moved inside the sublayer branch.
- **Point 2:** The student must identify that the main residual stream becomes purely additive (or acts as a direct identity path).
- **Point 3:** The student must connect the additive path to backpropagation, noting that the derivative of addition is 1, which preserves the magnitude of the gradients as they travel back through the layers.